

Claude 플러그인 4가지 상세 설명

네 가지 모두 Anthropic 공식 Claude Code 플러그인 마켓플레이스에 등록된 항목들입니다. 최상길님이 GD RDM 프로젝트에서 Claude Code를 쓰고 계시니, 실제로 유용할 만한 것들이라 하나씩 설명드릴게요.

1. session-report

Claude Code 세션이 끝난 뒤 사용 내역을 시각적으로 탐색할 수 있는 HTML 리포트를 생성해주는 플러그인입니다Generate an explorable HTML report of Claude Code session usage — tokens, cache efficiency, subagents, skills, and th.... 토큰 사용량, 캐시 효율, 서브에이전트 활용, 스킬 호출 내역 등을 한눈에 볼 수 있어서, 작업이 얼마나 효율적으로 진행됐는지(또는 토큰을 어디서 낭비했는지) 파악하는 데 도움이 됩니다. GD RDM처럼 대용량 CSV를 다루며 Schema-First 접근을 쓰는 프로젝트에서는, 어떤 단계에서 토큰이 많이 쓰였는지 확인하고 워크플로우를 최적화하는 데 참고하기 좋습니다.

2. claude-md-management

CLAUDE.md 파일의 품질을 점검하고, 세션에서 얻은 학습 내용을 CLAUDE.md(또는 .claude.local.md)에 반영하도록 도와주는 플러그인입니다Keep your project memory fresh and effective. This plugin provides tools to audit CLAUDE.md file quality and capture session learnings, ensuring Claude always has the context it needs to work efficiently in your codebase.

핵심 기능은 두 가지입니다. 첫째, claude-md-improver 스킬이 저장소 내 모든 CLAUDE.md 파일을 찾아서 명령어/아키텍처/주의사항/간결성 같은 기준으로 평가하고 등급이 포함된 품질 리포트를 생성하며, 부족한 부분에 대한 추가 항목을 제안합니다The claude-md-improver skill scans your repository for all CLAUDE.md files, evaluates them against quality criteria (commands, architecture, gotchas, conciseness), and generates a detailed quality report with scores and grades. It then proposes targeted additions based on gaps it discovers.

둘째, /revise-claude-md 명령은 세션 종료 시 발견한 bash 명령, 코드 패턴, 환경상 특이점 등을 바탕으로 적절한 CLAUDE.md나 .claude.local.md 파일에 업데이트할 내용을 제안합니다The /revise-claude-md command helps you capture learnings at the end of a session—bash commands discovered, code patterns followed, environment quirks encountered—and suggests updates to the appropriate CLAUDE.md or .claude.local.md file. 변경 사항은 diff 형태로 보여주고 사용자 승인 후에만 적용됩니다Run /revise-claude-md after a productive session to capture new insights. The plugin will show you

proposed changes as diffs and only apply them with your approval.

최상길님이 최근 CLAUDE.md 파일을 GD 프로젝트 연속성 관리용으로 쓰고 계신다고 하셨으니, 이 플러그인이 그 관리 작업을 자동화하는 데 직접적으로 도움이 될 것 같습니다.

3. serena

Language Server Protocol(LSP) 기반의 시맨틱 코드 분석 MCP 서버입니다. 단순 텍스트 검색이 아니라 "심볼" 단위로 코드를 이해하고 탐색·편집할 수 있게 해줍니다Semantic code analysis MCP server for intelligent code understanding, refactoring, and navigation via language server protocol. Serena transforms Claude into a powerful coding agent with IDE-like capabilities. This MCP server provides semantic code retrieval and editing tools that work at the symbol level, enabling intelligent navigation of large codebases without reading entire files. Python, JavaScript, TypeScript, Java, Rust, Go, C++ 등 30개 이상의 언어를 지원합니다Supporting over 30 programming languages including Python, JavaScript, TypeScript, Java, Rust, Go, and C++, Serena leverages Language Server Protocol integration to deliver precise code understanding.

대표적인 도구로 find_symbol(심볼 정의 찾기), find_referencing_symbols(특정 클래스/함수를 참조하는 모든 곳 찾기), insert_after_symbol(특정 메서드 뒤에 정밀하게 코드 삽입) 등이 있고, 전체 파일을 읽지 않고도 코드의 관계 구조를 활용해 토큰을 절약하면서 작업할 수 있습니다Key features include symbol-level code extraction with tools like find_symbol and find_referencing_symbols, precision editing operations such as insert_after_symbol, and token-efficient codebase interaction. The toolkit exploits the relational structure of code to help Claude work effectively on complex projects.

GD RDM 프로젝트는 rdm_data_generator.py, rdm_merge.py, rdm_analysis.py 등 여러 모듈로 구성되어 있고 앞으로 코드베이스가 더 커질 가능성이 있는데, 이런 상황에서 "이 함수가 어디서 호출되는지", "이 클래스를 수정하면 어디에 영향이 가는지" 같은 질문에 훨씬 정확하고 빠르게 답할 수 있게 해주는 도구입니다. 단, oraios 측 README에서는 마켓플레이스를 통한 설치보다 직접 설치를 권장하는 점도 참고하시면 좋습니다Do not install Serena via an MCP or plugin marketplace!

4. hookify

Claude Code의 Hooks 기능을 복잡한 JSON 설정 없이, 자연어나 마크다운 규칙 파일로 손쉽게 만들 수 있게 해주는 플러그인입니다Easily create custom hooks to prevent unwanted behaviors by analyzing conversation patterns or explicit instructions

규칙은 YAML 프론트매터가 있는 마크다운 파일로 작성되며, bash 명령, 파일 편집, 사용자 프롬프트, 세션 종료 등 다양한 이벤트 타입에 대해 정규식 패턴 매칭을 지원하고, 변경 사항은 재시작 없이 즉시 적용됩니다Rules are configured with YAML frontmatter in markdown files, supporting regex pattern matching across different event types: bash commands, file edits, user prompts, and session stops. Changes take effect immediately without restarting Claude.

사용 예시로는 `/hookify rm -rf` 명령을 쓸 때 경고해줘 처럼 자연어로 규칙을 만들거나, `/hookify`를 인자 없이 실행하면 최근 대화를 분석해서 사용자가 직접 교정했던 행동들을 바탕으로 규칙을 자동 생성해줍니다Create rules from natural language: `/hookify Warn me when I use rm -rf commands` or `/hookify Don't use console.log in TypeScript files`. Run `/hookify` without arguments to analyze recent conversation and auto-generate rules from behaviors you've corrected. `/hookify:list`로 활성화된 규칙을 확인하고 `/hookify:configure`로 관리할 수 있습니다Use `/hookify:list` to see active rules and `/hookify:configure` for interactive management.

예시 규칙 형식은 이렇습니다--- name: warn-sensitive-files enabled: true event: file action: warn conditions: - field: file_path operator: regex_match pattern: .env\$|credentials|secrets - field: new_text operator: contains pattern: KEY --- 🗝️
Sensitive file edit detected!

GD 프로젝트는 회사 보안 정책상 외부 AI 서비스 사용이 제한되고 사내 데이터 파일 외부 유출이 금지된 환경이라고 하셨으니, hookify로 "민감 파일(.env, credentials 등) 수정 시 경고" 또는 "특정 데이터 파일을 outputs 디렉토리 밖으로 옮기지 못하게 차단" 같은 규칙을 만들어두면 안전장치 역할을 할 수 있을 것 같습니다.

정리하면, `claude-md-management`와 `hookify`는 GD RDM 프로젝트의 "운영/관리" 측면 (프로젝트 메모리 유지, 안전 규칙)에, `serena`는 코드베이스가 커질 때의 "탐색/리팩토링" 측면에, `session-report`는 "사후 분석" 측면에 도움이 되는 조합입니다. 설치하는 `/plugin install {plugin-name}@claude-plugins-official` 형태로 진행하시면 됩니다.

GD RDM 프로젝트(C:\wclaude\projects\GD) 맥락에 맞춰서 각 플러그인을 실제로 언제, 어떻게 쓰면 좋을지 구체적인 시나리오로 설명드릴게요.

1. session-report — "이번 세션에서 토큰을 어디에 썼는지" 점검

설치: `/plugin install session-report@claude-plugins-official`

GD RDM은 542개 분석 대상 컬럼, 수천 행짜리 CSV를 다루는 프로젝트라서, Claude Code가 큰 파일을 직접 읽으면 토큰이 빠르게 소모됩니다. 다음과 같은 타이밍에 리포트를 뽑아보면 유용합니다.

- `rdm_data_generator.py`나 `rdm_analysis.py`에서 큰 기능 단위(예: Feature 3 → Feature 4 전환) 작업을 끝낸 직후, 세션 리포트를 생성해서 어느 단계에서 토큰이 많이 들었는지 확인합니다. 예를 들어 "샘플 파일을 직접 읽은 부분"이 많이 차지한다면, Schema-First 원칙(메타데이터/샘플만 전달)을 더 엄격히 적용해야 한다는 신호가 됩니다.
- 서브에이전트(subagent)나 스킬을 활용한 작업 후, 어떤 스킬이 자주 호출되는지 보고 불필요하게 자동 트리거되는 스킬이 있다면 제거를 고려합니다.
- 캐시 효율(cache efficiency) 지표를 보고, CLAUDE.md나 자주 참조하는 파일들이 프롬프트 캐시를 잘 활용하고 있는지 확인 — 만약 캐시 적중률이 낮다면 CLAUDE.md 구조를 정리해서 변경이 적은 부분과 자주 바뀌는 부분을 분리하는 게 좋습니다.

즉, "주기적으로 돌리는 도구"보다는 "큰 작업 단위 종료 후 회고용"으로 쓰는 게 효율적입니다.

2. claude-md-management — CLAUDE.md를 자동으로 최신 상태 유지

설치: `/plugin install claude-md-management@claude-plugins-official`

최상길님이 이미 CLAUDE.md를 프로젝트 연속성용으로 관리하고 계시니, 이 플러그인은 그 작업을 절반 자동화해줍니다.

세션 종료 시 워크플로우: 작업을 마칠 때 `/revise-claude-md`를 실행합니다. 그러면 이번 세션에서 발견된 내용 — 예를 들어 "`rdm_merge.py` 실행 시 인코딩이 cp949라 발생하는 오류와 해결법", "`sample_merged.csv`는 3,000행 stratified sample이므로 분석 시 전체 데이터와 비율이 다를 수 있음", "`os.makedirs()`로 폴더 자동 생성되므로 별도 `mkdir` 불필요" 같은 내용 — 을 diff 형태로 제안받고, 승인하면 CLAUDE.md(전역) 또는 `.claude.local.md`(로컬, Git에 안 올라가는 개인 메모)에 자동 반영됩니다.

월/주 단위 점검: claude-md-improver 스킬을 통해 "CLAUDE.md 품질 점검해줘" 같은 요청을 하면, 명령어 섹션·아키텍처 설명·주의사항(gotchas)·간결성 기준으로 등급을 매긴 리포트를 받습니다. 예를 들어 "Feature 1~4 아키텍처 설명은 있는데, Jupyter Lab → Claude Code 전환 시 주의점(가상환경 활성화 명령 등)이 빠져 있다" 같은 구체적인 보강 포인트를 제안받을 수 있습니다.

특히 Git/GitHub 워크플로우를 도입하신 만큼, "이번 커밋에서 바뀐 것 + 다음에 알아야 할 것"을 매 세션 끝에 자동 정리해두면, 다음 세션 시작 시 "이전 세션에서 어디까지 했지?"를 다시 설명할 필요가 줄어듭니다.

3. serena — 코드베이스가 커질 때 정밀 탐색/리팩토링

설치 (마켓플레이스 대신 직접 설치 권장):

```
cd C:\workspace\projects\GD
```

```
claude mcp add serena -- uvx --from git+https://github.com/oraios/serena serena start-mcp-server --context claude-code --project $(pwd)
```

```
uvx --from git+https://github.com/oraios/serena serena project create
```

GD 프로젝트에서 구체적으로 쓸 만한 상황:

- "rdm_analysis.py에서 정의된 모든 분석 함수가 dashboard 코드에서 어디서 호출되는지 찾아줘" → find_referencing_symbols로 전체 파일을 읽지 않고도 호출 위치를 정확히 짚어줍니다.
- "rdm_merge.py의 병합 로직 함수에 새로운 검증 단계를 추가하고 싶은데, 함수 정의 바로 뒤에 넣어줘" → insert_after_symbol로 정확한 위치에 코드 삽입.
- 4개 기능(생성기/병합/분석/대시보드)이 서로 어떻게 의존하는지 파악할 때 — 예를 들어 "deferred deduplication 로직이 어느 모듈에 있고 어디서 호출되는지" 같은 아키텍처 질문에 시맨틱 검색으로 빠르게 답을 얻을 수 있습니다.
- 대시보드 코드가 React/Vue 등으로 커질 경우, 컴포넌트 간 참조 관계 추적에도 유용합니다.

지금은 코드베이스가 비교적 작은 단계라 효과가 크지 않을 수 있지만, Feature 4(대시보드) 이후 파일 수가 늘어나고 모듈 간 의존성이 복잡해지면 진가를 발휘합니다. 처음에는 serena project index로 인덱싱부터 해두는 게 좋습니다.

4. hookify — 보안 정책을 코드 차원에서 강제

설치: `/plugin install hookify@claude-plugins-official`

회사 보안 정책(외부 AI 서비스 사용 금지, 사내 데이터 파일 외부 반출 금지)을 고려하면 다음과 같은 규칙들이 특히 유용합니다.

자연어로 규칙 생성:

/hookify 실제 사내 데이터(real_data 폴더 또는 *.csv 중 'synthetic'이 이름에 없는 파일)를 git add 하려고 하면 차단해줘

이러면 다음과 같은 형태의 규칙 파일이 생성됩니다:

name: block-real-data-commit

enabled: true

event: bash

conditions:

- field: command

operator: regex_match

pattern: git add.*\w.csv

- field: command

operator: regex_match

pattern: ^(?!\w.*synthetic).\w\$

action: block

 실제 데이터로 추정되는 CSV를 Git에 추가하려고 합니다. synthetic 데이터인지 확인하세요.

다른 활용 예:

- "환경변수나 API 키가 포함된 파일(.env, config 등)을 수정할 때 경고해줘" → 사내 인증 정보 실수 유출 방지.
- "C:\wclaude\projects\wGD 외부 경로로 파일을 복사/이동하려는 명령은 경고해줘" → 데이터 반출 정책 준수.
- 세션 종료(stop) 이벤트에 대해 "분석 코드를 수정했는데 sample_merged.csv로 테

스트를 실행하지 않았으면 멈추지 말라고 알려줘" 같은 품질 규칙도 가능합니다.

처음에는 /hookify를 인자 없이 실행해서, 최근 세션에서 직접 교정했던 행동들(예: "이 폴더에 쓰면 안 된다고 여러 번 말했던 것")을 기반으로 규칙을 자동 생성받아 보는 것도 좋은 출발점입니다. 생성된 규칙은 /hookify:list로 확인하고 /hookify:configure로 다듬으시면 됩니다.

조합 활용 시나리오: 세션 시작 → (serena로 필요한 코드 부분만 정밀 탐색) → 작업 수행 (hookify가 백그라운드에서 보안 규칙 감시) → 세션 종료 시 /revise-claude-md로 학습 내용 CLAUDE.md에 반영 → 가끔 session-report로 효율성 점검. 이렇게 묶으면 "다음 세션도 매끄럽게 이어가기"와 "보안 정책 준수"라는 GD 프로젝트의 두 핵심 과제를 동시에 챙길 수 있습니다.

짧게 답하면 "거의 맞지만 몇 가지 확인/수정이 필요한 부분이 있다"입니다. 항목별로 정리하면:

1. session-report, 2. claude-md-management, 4. hookify — 거의 그대로 됩니다

이 셋은 모두 claude-plugins-official 마켓플레이스에 속한 Anthropic 공식 플러그인이고, 이 마켓플레이스는 Claude Code 설치 시 기본으로 내장되어 있어서 별도로 /plugin marketplace add를 할 필요 없이 The official Anthropic marketplace (claude-plugins-official) is built in and available the moment you start Claude Code — you don't have to add it. Claude Code CLI 안에서 바로

```
/plugin install session-report@claude-plugins-official
```

```
/plugin install claude-md-management@claude-plugins-official
```

```
/plugin install hookify@claude-plugins-official
```

를 입력하면 설치됩니다 To install, run /plugin install {plugin-name}@claude-plugins-official 설치 후에는 /reload-plugins로 재로드하고, /plugin 명령으로 열리는 관리 화면의 Errors 탭에서 정상 로드됐는지 확인하면 좋습니다 Install it with /plugin install <plugin-name>@claude-plugins-official, then run /reload-plugins and check the Errors tab to confirm the binary is found.

다만 hookify에서 제가 보여드린 YAML 규칙 예시는 "이런 형태로 생성된다"는 걸 보여주는 예시일 뿐, 실제로는 /hookify <자연어 설명> 명령을 실행하면 플러그인이 자동으로 규칙 파일을 생성해줍니다 Create a rule from plain English /hookify Warn me when I use rm 직접 YAML을 손으로 작성할 필요는 없고, 자연어로 요청한 뒤 생성된 결과를 /hookify:list로 확인·조정하시면 됩니다.

3. serena — 좀 더 신경 써야 할 부분이 있습니다

두 가지 경로가 있는데, 결과가 약간 다릅니다.

(a) 마켓플레이스 경로: claude-plugins-official에도 Serena가 등록되어 있어서 /plugin install serena@claude-plugins-official 식으로 설치됩니다.

(b) oraios(serena 원작자) 직접 설치 경로: 원작자 GitHub README에는 "마켓플레이스 /MCP 경유 설치하지 마라"는 경고가 명시되어 있습니다 Do not install Serena via an MCP or plugin marketplace! 즉, 마켓플레이스 버전은 업데이트가 늦거나 일부 설정이 다를 수 있으니, 최신 기능을 안정적으로 쓰려면 제가 앞서 드린 `uvx --from git+https://github.com/oraios/serena ...` 명령으로 직접 MCP 서버를 등록하는 방식을 권장한다는 의미입니다.

이때 주의할 점:

- uvx는 Python 패키지 매니저 uv에 포함된 명령이므로, Windows 환경에 uv가 먼저 설치되어 있어야 합니다 (pip install uv 또는 공식 설치 스크립트).
- 이 명령은 cd C:\claude\projects\WGD 같은 프로젝트 루트에서 실행해야 --project \$(pwd) 부분이 올바르게 잡힙니다. PowerShell에서는 \$(pwd) 대신 \${PWD} 또는 그냥 절대경로를 직접 써주는 게 안전합니다.
- claude mcp add 명령은 Claude Code CLI(터미널)에서 실행하는 것이고, /plugin install처럼 Claude Code 대화창 안에서 치는 명령(슬래시 명령)이 아닙니다 — 둘을 구분해서 실행하셔야 합니다.

회사 보안 정책 관련 추가 확인사항

GD 프로젝트는 외부 AI 서비스 사용 제한, 사내 데이터 외부 반출 제한이 있는 환경이라고 하셨으니, serena처럼 외부 GitHub 저장소(oraio/serena)에서 코드를 받아 로컬에서 실행하는 도구나, hookify-claude-md-management처럼 자동으로 파일을 수정/생성하는 플러그인은 설치 전에 회사 보안팀 정책상 "신규 외부 패키지 설치"나 "MCP 서버 실행"이 허용되는지 한 번 확인해보시는 게 안전합니다. 플러그인은 사용자 권한으로 임의 코드를 실행할 수 있다는 점도 참고하시면 좋습니다Plugins can execute arbitrary code with your user privileges. Treat a marketplace like you'd treat installing an npm package — trust the source, read what's in it if it matters.

정리하면, session-report / claude-md-management / hookify는 위에서 설명한 대로 거의 그대로 실행하시면 되고, serena만 Windows 환경 설정(uv 설치, 경로 표기)과 사내 보안 검토를 한 번 더 확인하고 진행하시는 걸 권장합니다.

GD RDM 프로젝트의 작업 패턴(Python/CSV 분석, Windows, Git/GitHub, 세션 연속성 관리, 보안 제약, Feature 1~4 구조, 웹 대시보드)과 최상길님의 작업 스타일(원리 이해 선호, Schema-First 접근)을 고려해서 추천드립니다.

1. Pyright LSP — Python 코드 품질

```
/plugin install pyright-lsp@claude-plugins-official
```

rdm_data_generator.py, rdm_merge.py, rdm_analysis.py 같은 파일들이 점점 커지고 있는데, Pyright 기반 언어 서버를 연결하면 Claude가 타입 체크, 정의로 이동, 미사용 변수 /import 같은 문제를 코드 작성 시점에 바로 잡아줍니다. 542개 컬럼을 다루는 코드에서 컬럼명 오타나 타입 불일치 같은 실수를 줄이는 데 특히 유용합니다. serena와 역할이 겹치는 부분도 있지만, Pyright는 더 가볍고 Python 전용이라 우선 도입하기 좋습니다.

2. Security Guidance — 보안 정책 자동 점검

```
/plugin install security-guidance@claude-plugins-official
```

Claude가 코드를 작성할 때마다 명령어 삽입, 하드코딩된 자격 증명, 안전하지 않은 파일 경로 처리 같은 문제를 자동으로 검토하고 같은 세션 내에서 수정하도록 유도하는 확입니다The security-guidance plugin reviews each change Claude makes for common vulnerabilities and instructs Claude to fix what it finds in the same session. 회사의 "사내 데이터 파일 외부 반출 금지" 정책과 직접 연결되진 않지만 (그건 hookify로 커스텀 규칙을 만들어야 함), 일반적인 코드 보안 위생 관리에는 hookify와 함께 기본적으로 깔아둘 만합니다.

3. Remember — 세션 연속성 (CLAUDE.md 관리의 보완재)

```
/plugin install remember@claude-plugins-official
```

대화 내용을 추출/요약해서 계층화된 일일 로그로 압축 저장하는 연속 메모리 플러그인입니다Continuous memory for Claude Code: extracts, summarizes, compresses conversations into tiered daily logs. claude-md-management은 "프로젝트 구조/규칙"을 관리하는 데 강하고, remember는 "오늘 무슨 작업을 했고 어디까지 진행했는지"를 시간 순으로 누적하는 데 강합니다. 세션 재개가 자주 필요한 GD 프로젝트 특성상, 두 플러그인을 함께 쓰면 "이전 세션에서 어디까지 했지?"라는 질문에 대한 답을 CLAUDE.md(구조/규칙)와 일일 로그(작업 흐름) 양쪽에서 얻을 수 있습니다.

4. Commit Commands — Git 워크플로우 단축

```
/plugin install commit-commands@claude-plugins-official
```

최근 Git/GitHub 설정을 막 끝내신 만큼, 이 플러그인의 `/commit-commands:commit` 같은 명령으로 변경사항 요약 → 의미있는 커밋 메시지 작성 → 커밋/푸시까지 한 번에 처리할 수 있습니다. Feature 단위로 작업하시는 패턴이라 "Feature 3 완료 → 자동 커밋 메시지 생성"처럼 쓰기 좋습니다.

5. Feature Dev — Feature 단위 개발 워크플로우

```
/plugin install feature-dev@claude-plugins-official
```

탐색(code-explorer)→설계(code-architect)→리뷰(code-reviewer) 에이전트로 구성된 가이드형 개발 워크플로우입니다 Command: `/feature-dev` - Guided feature development workflow Agents: code-explorer, code-architect, code-reviewer - For codebase analysis, architecture design, and quality review GD 프로젝트가 이미 "Feature 1(생성기) → Feature 2(병합) → Feature 3(분석) → Feature 4(대시보드)"라는 명확한 단계 구조를 갖고 있으니, 다음 Feature 작업을 시작할 때 `/feature-dev`로 탐색/설계/리뷰를 구조화하면 즉흥적인 구현보다 일관성 있는 결과를 얻을 수 있습니다.

6. Frontend Design — 웹 대시보드(Feature 4)용

```
/plugin install frontend-design@claude-plugins-official
```

Feature 4 대시보드 작업 시 자동으로 트리거되어, 일반적인 "AI스러운" 디자인 대신 더 정제된 타이포그래피·레이아웃·인터랙션을 제안해주는 스킴입니다. 대시보드가 분석 결과를 보여주는 핵심 인터페이스가 될 테니, 처음부터 이 스킴을 켜두면 나중에 디자인을 다시 손보는 수고를 줄일 수 있습니다.

7. Playground — 데이터 탐색/프로토타이핑

```
/plugin install playground@claude-plugins-official
```

슬라이더, 토글 같은 비주얼 컨트롤이 포함된 인터랙티브 HTML 프로토타입을 빠르게 만들어주는 플러그인입니다. "542개 컬럼 중 특정 그룹만 필터링해서 분포를 보고 싶다" 같은 1회성 데이터 탐색이나, 대시보드 설계 전 와이어프레임 검토용으로 활용할 수 있습니다.

8. Playwright (선택) — Naver Cafe 다운로더 보완

```
/plugin install playwright@claude-plugins-official
```

지금은 Selenium으로 구현된 Naver Cafe 이미지 다운로더(cafe_main iframe, club ID 14942836)를 Claude Code 안에서 디버깅하거나, 비슷한 JS 렌더링 페이지를 다룰 새 자동화를 만들 때 Playwright MCP를 쓰면 Claude가 직접 브라우저를 띄워 화면 상태를 확

인하면서 코드를 작성/디버그할 수 있습니다. 핵심 업무는 아니지만 개인 프로젝트 쪽에서 유용합니다.

플러그인이 아닌 "직접 만드는 스킬" 제안

위 기성 플러그인들과 별개로, Skill Creator 플러그인(/plugin install skill-creator@claude-plugins-official)을 이용해 GD 프로젝트 전용 커스텀 스킬을 하나 만들어두는 것도 추천드립니다. 예를 들어 "rdm-schema-first-analysis"라는 스킬을 만들어, "이 프로젝트에서는 원본 CSV를 직접 읽지 말고 항상 메타데이터/샘플(sample_merged.csv)만 참조할 것", "542개 컬럼 명명 규칙은 이것이다", "deduplication은 자연 처리한다" 같은 GD 프로젝트 고유의 규칙을 스킬로 인코딩해두면, CLAUDE.md보다 더 명시적으로 "이 작업 유형에서는 이렇게 행동하라"는 가이드를 Claude에게 줄 수 있습니다. claude-md-management가 CLAUDE.md를 관리해주는 것과 상호보완적으로 쓰면 좋습니다.

우선순위 제안: 지금 단계에서는 pyright-lsp + commit-commands + security-guidance 정도를 먼저 깔아서 일상 작업 품질을 높이고, Feature 4(대시보드) 작업에 들어갈 때 frontend-design + playground를, 그리고 세션 재개 문제가 계속 번거롭다면 remember를 추가하는 순서가 자연스러울 것 같습니다.